

MVJPA-RS-0626 專案學習文件

技術棧：Jakarta EE 10 + JAX-RS (Jersey) + JPA (Hibernate) + MySQL + Jackson JSON 開

發工具：Eclipse + Maven + Tomcat 10.1 + Postman

一、專案結構概覽

```
mvjpars0626/
├── pom.xml                                # Maven 建置設定
├── src/
│   ├── main/
│   │   ├── java/
│   │   │   ├── config/
│   │   │   │   ├── JaxrsApplication.java    # JAX-RS 應用入口
│   │   │   │   ├── JpaUtil.java             # JPA EntityManager 工具
│   │   │   │   ├── JacksonConfig.java        # Jackson JSON 序列化設定
│   │   │   │   └── EmployeeController.java    # REST API Controller
│   │   │   ├── model/
│   │   │   │   ├── Employee.java             # Entity 實體類別
│   │   │   │   └── repository/
│   │   │   │       ├── MyRepository.java      # 泛型 Repository 介面
│   │   │   │       └── EmployeeRepository.java # Employee DAO 實作
│   │   │   ├── resources/
│   │   │   │   ├── META-INF/
│   │   │   │   │   └── persistence.xml        # JPA 持久化設定
│   │   │   └── webapp/
│   │   │       ├── WEB-INF/
│   │   │       │   └── web.xml                 # Web 部署描述檔
│   │   └── test/
│   │       └── (測試程式可自行補充)
└── target/                                # Maven 編譯輸出
```

二、pom.xml：Maven 建置設定解析

2.1 專案基本資訊

```
<groupId>demo</groupId>
<artifactId>mvjpars0626</artifactId>
<version>0.0.1-SNAPSHOT</version>
<packaging>war</packaging>
```

元素	說明
<code>groupId</code>	組織識別碼，類似套件命名空間
<code>artifactId</code>	專案識別碼，會作為 WAR 檔名基底
<code>version</code>	版本號， <code>SNAPSHOT</code> 表開發中版本
<code>packaging</code>	<code>war</code> → 打包成 Web Archive，部署至 Servlet 容器 (Tomcat)

2.2 屬性定義

```
<properties>
  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  <maven.compiler.source>21</maven.compiler.source>
  <maven.compiler.target>21</maven.compiler.target>
  <eclipselink.version>4.0.2</eclipselink.version>
  <jackson.version>2.16.1</jackson.version>
  <jersey.version>3.1.6</jersey.version>
</properties>
```

屬性	值	用途
<code>maven.compiler.source</code> / <code>target</code>	21	Java 版本設定
<code>eclipselink.version</code>	4.0.2	EclipseLink (JPA 實作) 版本 (但實際使用 Hibernate)
<code>jackson.version</code>	2.16.1	Jackson JSON 處理套件版本
<code>jersey.version</code>	3.1.6	Jersey (JAX-RS 實作) 版本

2.3 相依套件詳解

(1) Servlet / JSP / JSTL (Web 容器基礎)

```
<!-- Servlet API (Jakarta EE 10 / Tomcat 10.1) -->
<dependency>
  <groupId>jakarta.servlet</groupId>
  <artifactId>jakarta.servlet-api</artifactId>
  <version>6.0.0</version>
  <scope>provided</scope>
</dependency>

<!-- JSP API -->
<dependency>
  <groupId>jakarta.servlet.jsp</groupId>
  <artifactId>jakarta.servlet.jsp-api</artifactId>
  <version>3.1.0</version>
  <scope>provided</scope>
</dependency>

<!-- JSTL (含 API 與實作) -->
<dependency>
  <groupId>jakarta.servlet.jsp.jstl</groupId>
  <artifactId>jakarta.servlet.jsp.jstl-api</artifactId>
  <version>3.0.0</version>
</dependency>
```

套件	用途	Scope
<code>jakarta.servlet-api</code> 6.0.0	Servlet API	<code>provided</code> (Tomcat 提供)
<code>jakarta.servlet.jsp-api</code> 3.1.0	JSP API	<code>provided</code> (Tomcat 提供)
<code>jakarta.servlet.jsp.jstl-api</code> 3.0.0	JSTL 標籤庫	一般相依

`scope=provided` : 部署時由 Tomcat 提供，避免與容器內建 jar 衝突。

(2) JAX-RS + Jersey (REST API 框架)

```
<!-- JAX-RS API (Jakarta EE 10 / Tomcat 10.1) -->
<dependency>
  <groupId>jakarta.ws.rs</groupId>
  <artifactId>jakarta.ws.rs-api</artifactId>
  <version>3.1.0</version>
</dependency>

<!-- Jersey Core Server -->
<dependency>
  <groupId>org.glassfish.jersey.core</groupId>
  <artifactId>jersey-server</artifactId>
  <version>${jersey.version}</version>
</dependency>

<!-- Jersey Servlet Container -->
<dependency>
  <groupId>org.glassfish.jersey.containers</groupId>
  <artifactId>jersey-container-servlet</artifactId>
  <version>${jersey.version}</version>
</dependency>

<!-- Jersey HK2 Injection -->
<dependency>
  <groupId>org.glassfish.jersey.inject</groupId>
  <artifactId>jersey-hk2</artifactId>
  <version>${jersey.version}</version>
</dependency>
<!-- JSON 支援 (Jackson) -->
<dependency>
  <groupId>org.glassfish.jersey.media</groupId>
  <artifactId>jersey-media-json-jackson</artifactId>
  <version>${jersey.version}</version>
</dependency>
```

套件	用途
<code>jakarta.ws.rs-api</code> 3.1.0	JAX-RS 標準 API
<code>jersey-server</code> 3.1.6	Jersey 核心伺服器
<code>jersey-container-servlet</code> 3.1.6	Jersey Servlet 容器整合
<code>jersey-hk2</code> 3.1.6	HK2 依賴注入支援
<code>jersey-media-json-jackson</code> 3.1.6	Jersey + Jackson JSON 整合

(3) Jackson JSON 處理

```
<dependency>
  <groupId>com.fasterxml.jackson.module</groupId>
  <artifactId>jackson-module-jakarta-xmlbind-
annotations</artifactId>
  <version>2.17.0</version>
</dependency>
<!-- Java 8+ 日期模組 -->
<dependency>
  <groupId>com.fasterxml.jackson.datatype</groupId>
  <artifactId>jackson-datatype-jsr310</artifactId>
  <version>2.16.1</version>
</dependency>
```

套件	用途
<code>jackson-module-jakarta-xmlbind-annotations</code> 2.17.0	支援 Jakarta XML Binding 註解
<code>jackson-datatype-jsr310</code> 2.16.1	Java 8 日期時間 (LocalDate, LocalDateTime) 序列化

(4) JPA + Hibernate (資料持久層)

```
<dependency>
  <groupId>org.hibernate.orm</groupId>
  <artifactId>hibernate-core</artifactId>
  <version>6.6.1.Final</version>
</dependency>
<dependency>
  <groupId>org.hibernate.orm</groupId>
  <artifactId>hibernate-hikaricp</artifactId>
  <version>6.6.1.Final</version>
</dependency>

<dependency>
  <groupId>com.mysql</groupId>
  <artifactId>mysql-connector-j</artifactId>
  <version>9.2.0</version>
</dependency>
```

套件	用途
<code>hibernate-core</code> 6.6.1.Final	Hibernate ORM 核心 (JPA 實作)

套件	用途
<code>hibernate-hikaricp</code> 6.6.1.Final	HikariCP 連線池整合 · 提升資料庫連線效能

(5) MySQL JDBC 驅動

套件	用途
<code>mysql-connector-j</code> 9.2.0	MySQL JDBC 驅動程式 (Java 9+ 適用)

(6) 測試

套件	用途	Scope
<code>junit</code> 4.13.1	單元測試	<code>test</code>

2.4 Build 設定

```
<finalName>mvjpars0626</finalName>
```

將 WAR 檔命名為 `mvjpars0626.war` · 部署路徑即為 `/mvjpars0626` 。

主要外掛：

- `maven-compiler-plugin` 3.13.0 : 編譯 Java 21 程式
- `maven-war-plugin` 3.4.0 : 打包 WAR
- `maven-surefire-plugin` 3.3.0 : 執行測試

三、persistence.xml：JPA 持久化設定解析

```
<?xml version="1.0" encoding="UTF-8"?>
<persistence version="3.0" xmlns="https://jakarta.ee/xml/ns/persistence"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="https://jakarta.ee/xml/ns/persistence
https://jakarta.ee/xml/ns/persistence/persistence_3_0.xsd">

    <persistence-unit name="jaxrsPU" transaction-type="RESOURCE_LOCAL">
        <class>model.Employee</class>
        <properties>
            <property name="jakarta.persistence.jdbc.driver"
                value="com.mysql.cj.jdbc.Driver"/>
            <property name="jakarta.persistence.jdbc.url"
                value="jdbc:mysql://localhost:3306/jaxrs_demo?
useSSL=false&serverTimezone=Asia/Taipei"/>
            <property name="jakarta.persistence.jdbc.user" value="root"/>
            <property name="jakarta.persistence.jdbc.password"
value="1234"/>
        </properties>
    </persistence-unit>
</persistence>
```

設定項目說明

項目	值	說明
version	3.0	Jakarta Persistence 3.0 (Jakarta EE 10)
persistence-unit name	jaxrsPU	持久化單元名稱・JpaUtil.java 依此名稱建立 EntityManagerFactory
transaction-type	RESOURCE_LOCAL	手動管理交易 (非 JTA 容器管理交易)
class	model.Employee	註冊 Entity 類別・等同於 @Entity 掃描

連線設定

Property	說明
jakarta.persistence.jdbc.driver	com.mysql.cj.jdbc.Driver (MySQL 8+ 驅動)
jakarta.persistence.jdbc.url	連線 URL：資料庫 jaxrs_demo・關閉 SSL・時區臺北
jakarta.persistence.jdbc.user	使用者 root

Property	說明
<code>jakarta.persistence.jdbc.password</code>	密碼 1234

注意：使用前須先在 MySQL 建立 `jaxrs_demo` 資料庫與 `employees` 表格，或設定 Hibernate 的 `ddl-auto` 自動建表（此專案未設定，需手動建表）。

建議建立的 SQL

```
CREATE DATABASE IF NOT EXISTS jaxrs_demo
  DEFAULT CHARACTER SET utf8mb4
  DEFAULT COLLATE utf8mb4_unicode_ci;

USE jaxrs_demo;

CREATE TABLE employees (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  name        VARCHAR(100)  NOT NULL,
  email       VARCHAR(150)  NOT NULL UNIQUE,
  department  VARCHAR(50)   NOT NULL,
  salary      DOUBLE,
  hire_date   DATE,
  created_at  DATETIME,
  updated_at  DATETIME
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- 測試資料
INSERT INTO employees (name, email, department, salary, hire_date,
  created_at, updated_at) VALUES
('Alice Chen', 'alice@example.com', 'Engineering', 85000, '2020-03-15', NOW(), NOW()),
('Bob Wang', 'bob@example.com', 'Marketing', 72000, '2021-07-01', NOW(), NOW()),
('Carol Lin', 'carol@example.com', 'Engineering', 95000, '2019-11-20', NOW(), NOW()),
('David Lee', 'david@example.com', 'HR', 65000, '2022-01-10', NOW(), NOW()),
('Eva Wu', 'eva@example.com', 'Marketing', 78000, '2022-06-15', NOW(), NOW());
```


四、web.xml：Web 部署描述檔解析

```
<web-app version="6.0"
  xmlns="https://jakarta.ee/xml/ns/jakartaee"
  ...>
  <display-name>Archetype Created Web Application</display-name>
</web-app>
```

- **version="6.0"**：符合 Jakarta EE 10 / Tomcat 10.1 規格
- 由於 JAX-RS 使用 `@ApplicationPath("/api")` 註解自動註冊，**web.xml** 不需要手動設定 **Servlet**，由 Servlet 3.0+ 的 SPI 機制自動掃描

五、各 Java 程式詳細說明

5.1 `config/JaxrsApplication.java` — JAX-RS 應用入口

```
@ApplicationPath("/api")
public class JaxrsApplication extends Application {
}
```

功能

項目	說明
<code>@ApplicationPath("/api")</code>	定義 REST API 基礎路徑為 <code>/api</code> ，所有 API 都以此開頭
繼承 <code>Application</code>	JAX-RS 應用程式起點
空類別	自動掃描同 War 中所有 <code>@Path</code> 與 <code>@Provider</code> 註解類別

完整 API 路徑範例

```
http://localhost:8080/mvjpars0626/api/employees
└─context└─┬─@ApplicationPath└─┬─@Path└─
```

5.2 config/JpaUtil.java — JPA EntityManager 工廠

```
public class JpaUtil {  
    private static final EntityManagerFactory emf;  
  
    static {  
        emf = Persistence.createEntityManagerFactory("jaxrsPU");  
    }  
  
    public static EntityManager createEntityManager() {  
        return emf.createEntityManager();  
    }  
  
    public static void close() {  
        if (emf != null && emf.isOpen()) emf.close();  
    }  
}
```

功能說明

方法	說明
<code>static</code> 初始區塊	類別載入時即建立 <code>EntityManagerFactory</code> (單例模式)
<code>Persistence.createEntityManagerFactory("jaxrsPU")</code>	讀取 <code>persistence.xml</code> 中 <code>name="jaxrsPU"</code> 的設定
<code>createEntityManager()</code>	每次資料庫操作取得一個新的 <code>EntityManager</code> (輕量級)
<code>close()</code>	應用程式關閉時釋放 <code>EntityManagerFactory</code> 資源
設計要點： <code>EntityManagerFactory</code> 建立成本高，應為單例； <code>EntityManager</code> 建立成本低，應每次請求建立並用完即關。	

5.3 config/JacksonConfig.java — JSON 序列化設定

```
@Provider
public class JacksonConfig implements ContextResolver<ObjectMapper> {
    private final ObjectMapper mapper;

    public JacksonConfig() {
        mapper = new ObjectMapper();
        mapper.registerModule(new JavaTimeModule());
        mapper.disable(SerializationFeature.WRITE_DATES_AS_TIMESTAMPS);
        mapper.disable(DeserializationFeature.FAIL_ON_UNKNOWN_PROPERTIES);
        mapper.setSerializationInclusion(JsonInclude.Include.NON_NULL);

        mapper.setPropertyNamingStrategy(PropertyNamingStrategies.LOWER_CAMEL_CASE);
    }

    @Override
    public ObjectMapper getContext(Class<?> type) {
        return mapper;
    }
}
```

各項設定說明

設定	效果
<code>@Provider</code>	註冊為 JAX-RS 提供者，Jersey 自動使用此 <code>ObjectMapper</code>
<code>JavaTimeModule</code>	支援 <code>LocalDate</code> 、 <code>LocalDateTime</code> 序列化
<code>disable(WRITE_DATES_AS_TIMESTAMPS)</code>	日期以 ISO-8601 字串輸出而非時間戳
<code>disable(FAIL_ON_UNKNOWN_PROPERTIES)</code>	JSON 有多餘欄位時不拋錯
<code>setSerializationInclusion(NON_NULL)</code>	序列化時忽略 <code>null</code> 欄位
<code>PropertyNamingStrategies.LOWER_CAMEL_CASE</code>	使用駝峰命名 (預設即為此)

5.4 model/Employee.java — JPA Entity 實體

Entity 映射

```
@Entity
@Table(name = "employees")
public class Employee {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;

    @Column(name = "name", nullable = false, length = 100)
    private String name;

    @Column(name = "email", nullable = false, unique = true, length = 150)
    private String email;

    @Column(name = "department", nullable = false, length = 50)
    private String department;

    @Column(name = "salary")
    private Double salary;

    @Column(name = "hire_date")
    @JsonFormat(pattern = "yyyy-MM-dd")
    private LocalDate hireDate;

    @Column(name = "created_at", updatable = false)
    @JsonFormat(pattern = "yyyy-MM-dd HH:mm:ss")
    private LocalDateTime createdAt;

    @Column(name = "updated_at")
    @JsonFormat(pattern = "yyyy-MM-dd HH:mm:ss")
    private LocalDateTime updatedAt;
}
```

欄位說明

欄位	Java 型態	SQL 型態	限制
id	Integer	INT	PK、自動遞增
name	String	VARCHAR(100)	NOT NULL
email	String	VARCHAR(150)	NOT NULL、UNIQUE
department	String	VARCHAR(50)	NOT NULL

欄位	Java 型態	SQL 型態	限制
salary	Double	DOUBLE	可為 NULL
hireDate	LocalDate	DATE	@JsonFormat("yyyy-MM-dd")
createdAt	LocalDateTime	DATETIME	updatable=false (創建後不更新)
updatedAt	LocalDateTime	DATETIME	每次更新自動修改

生命週期回呼 (Lifecycle Callbacks)

```
@PrePersist
protected void onCreate() {
    createdAt = LocalDateTime.now();
    updatedAt = LocalDateTime.now();
}

@PreUpdate
protected void onUpdate() {
    updatedAt = LocalDateTime.now();
}
```

回呼	觸發時機	動作
@PrePersist	em.persist() 前	自動填入 createdAt 、 updatedAt
@PreUpdate	em.merge() 前	自動更新 updatedAt

JSON 日期格式

欄位	格式	範例
hireDate	yyyy-MM-dd	"2024-06-26"
createdAt / updatedAt	yyyy-MM-dd HH:mm:ss	"2024-06-26 14:30:00"

5.5 repository/MyRepository.java — 泛型 Repository 介面

```
public interface MyRepository<T, ID> {  
    T save(T entity);  
    Optional<T> findById(ID id);  
    List<T> findAll();  
    T update(T entity);  
    void deleteById(ID id);  
    boolean existsById(ID id);  
}
```

泛型設計

型別參數	用途	EmployeeRepository 綁定
T	Entity 型別	Employee
ID	主鍵型別	Integer

方法定義

方法	對應 SQL	HTTP 對照
save(T)	INSERT	POST
findById(ID)	SELECT ... WHERE id=?	GET /{id}
findAll()	SELECT *	GET
update(T)	UPDATE	PUT /{id}
deleteById(ID)	DELETE WHERE id=?	DELETE /{id}
existsById(ID)	SELECT COUNT(*) WHERE id=?	-

5.6 repository/EmployeeRepository.java — Employee DAO 實作

save — 新增 (INSERT)

```
public Employee save(Employee emp) {  
    EntityManager em = JpaUtil.createEntityManager();  
    EntityTransaction tx = em.getTransaction();  
    try {  
        tx.begin();  
        em.persist(emp);        // INSERT INTO employees ...  
        tx.commit();  
        return emp;              // emp 的 id 會被自動填入  
    } catch (Exception e) {  
        if (tx.isActive()) tx.rollback();  
        throw e;  
    } finally {  
        em.close();  
    }  
}
```

重點： `persist()` 後，Hibernate 會自動將資料庫生成的 `id` 寫回 `emp` 物件的 `id` 屬性。

findById — 查詢單筆 (SELECT)

```
public Optional<Employee> findById(Integer id) {  
    EntityManager em = JpaUtil.createEntityManager();  
    try {  
        return Optional.ofNullable(em.find(Employee.class, id));  
    } finally {  
        em.close();  
    }  
}
```

重點： `em.find()` 若無資料回傳 `null`，包裝為 `Optional` 避免 `NullPointerException`。

findAll — 查詢全部 (SELECT ALL)

```
public List<Employee> findAll() {
    EntityManager em = JpaUtil.createEntityManager();
    try {
        return em.createQuery(
            "SELECT e FROM Employee e ORDER BY e.id",
            Employee.class).getResultList();
    } finally {
        em.close();
    }
}
```

重點：使用 JPQL (Java Persistence Query Language) · `Employee` 是 Entity 名稱而非表格名稱。

update — 更新 (UPDATE)

```
public Employee update(Employee emp) {
    EntityManager em = JpaUtil.createEntityManager();
    EntityTransaction tx = em.getTransaction();
    try {
        tx.begin();
        Employee merged = em.merge(emp); // UPDATE employees SET ...
        tx.commit();
        return merged; // 回傳受管理的 entity
    } catch (Exception e) {
        if (tx.isActive()) tx.rollback();
        throw e;
    } finally {
        em.close();
    }
}
```

重點： `merge()` 與 `persist()` 的差異 — `merge` 用於更新已存在的 entity · 可接收 detached entity 並回傳 managed entity。

deleteById — 刪除 (DELETE)

```
public void deleteById(Integer id) {
    EntityManager em = JpaUtil.createEntityManager();
    EntityTransaction tx = em.getTransaction();
    try {
        tx.begin();
        Employee emp = em.find(Employee.class, id);
        if (emp != null) em.remove(emp); // DELETE FROM employees WHERE
id=?
        tx.commit();
    } catch (Exception e) {
        if (tx.isActive()) tx.rollback();
        throw e;
    } finally {
        em.close();
    }
}
```

重點：JPA 刪除前須先 `find()` 取得 managed entity，才能 `remove()`。

findByDepartment — 依部門查詢

```
public List<Employee> findByDepartment(String dept) {
    return em.createQuery(
        "SELECT e FROM Employee e WHERE LOWER(e.department) =
LOWER(:dept) ORDER BY e.name",
        Employee.class)
        .setParameter("dept", dept)
        .getResultList();
}
```

重點：`:dept` 為命名參數 (named parameter)，避免 SQL Injection；`LOWER()` 確保不分大小寫比對。

findAllPaged — 分頁查詢

```
public List<Employee> findAllPaged(int page, int size) {
    return em.createQuery("SELECT e FROM Employee e ORDER BY e.id",
Employee.class)
        .setFirstResult((page - 1) * size) // OFFSET
        .setMaxResults(size) // LIMIT
        .getResultList();
}
```

重點：`setFirstResult` = OFFSET · `setMaxResults` = LIMIT · page 從 1 開始。

5.7 `config/EmployeeController.java` — REST API Controller

類別層級註解

```
@Path("/employees")           // API 路徑: /api/employees
@Produces(MediaType.APPLICATION_JSON) // 回傳 JSON
@Consumes(MediaType.APPLICATION_JSON) // 接收 JSON
public class EmployeeController {
    private final EmployeeRepository repo = new EmployeeRepository();
}
```

API 方法對照表

HTTP	路徑	Controller 方法	Repository 方法	說明
GET	<code>/api/employees</code>	<code>getAll()</code>	<code>findAllPaged()</code> / <code>findByDepartment()</code>	查詢全部 (可帶參數)
GET	<code>/api/employees/{id}</code>	<code>getId()</code>	<code>findById()</code>	查詢單筆
POST	<code>/api/employees</code>	<code>create()</code>	<code>save()</code>	新增
PUT	<code>/api/employees/{id}</code>	<code>update()</code>	<code>update()</code>	修改
DELETE	<code>/api/employees/{id}</code>	<code>delete()</code>	<code>deleteById()</code>	刪除

(1) GET `/api/employees` — 查詢全部 (含分頁與部門篩選)

```
@GET
public Response getAll(
    @QueryParam("dept") String dept,           // ?dept=IT
    @DefaultValue("1") @QueryParam("page") int page, // ?page=2
    @DefaultValue("10") @QueryParam("size") int size // ?size=5
) {
    if (dept != null) {
        return Response.ok(apiOk(repo.findByDepartment(dept))).build();
    }
    return Response.ok(apiOk(repo.findAllPaged(page, size))).build();
}
```

Query 參數	型態	預設值	範例
dept	String	null (可省略)	?dept=IT
page	int	1	?page=2&size=5
size	int	10	?size=20

回傳格式範例：

```
{
  "success": true,
  "data": [
    { "id": 1, "name": "John", "email": "john@test.com", ... },
    { "id": 2, "name": "Jane", "email": "jane@test.com", ... }
  ]
}
```

(2) GET /api/employees/{id} — 查詢單筆

```
@GET
@Path("/{id}")
public Response getById(@PathParam("id") int id) {
    return repo.findById(id)
        .map(emp -> Response.ok(apiOk(emp)).build())
        .orElse(Response.status(404).entity(apiError("Not found: " +
id)).build());
}
```

情境	HTTP 狀態碼	Body
找到	200	{"success":true,"data":{...}}
找不到	404	{"success":false,"error":"Not found: 99"}

(3) POST /api/employees — 新增

```
@POST
public Response create(Employee emp, @Context UriInfo uriInfo) {
    if (emp.getName() == null || emp.getName().isBlank()) {
        return Response.status(400).entity(apiError("Name is
required")).build();
    }
    Employee created = repo.save(emp);
    URI location = uriInfo.getAbsolutePathBuilder()
        .path(String.valueOf(created.getId())).build();
    return Response.created(location).entity(apiOk(created)).build();
}
```

驗證

不符合時

`name` 不得為空

400 Bad Request

情境	HTTP 狀態碼	Header	Body
成功	201 Created	<code>Location:</code> <code>.../api/employees/{id}</code>	<code>{"success":true,"data":{...}}</code>
驗證 失敗	400 Bad Request	-	<code>{"success":false,"error":"Name is required"}</code>

(4) PUT /api/employees/{id} — 修改

```
@PUT
@Path("/{id}")
public Response update(@PathParam("id") int id, Employee emp) {
    emp.setId(id); // 從 path 取出 id 設入 entity
    Employee updated = repo.update(emp);
    return Response.ok(apiOk(updated)).build();
}
```

回傳：200 OK · `{"success":true,"data":{"updated entity"}}`

(5) DELETE /api/employees/{id} — 刪除

```
@DELETE
@Path("/{id}")
public Response delete(@PathParam("id") int id) {
    repo.deleteById(id);
    return Response.noContent().build();
}
```

回傳：204 No Content (無 Body)

統一回應格式

```
private Map<String, Object> apiOk(Object data) {
    return Map.of("success", true, "data", data);
}

private Map<String, Object> apiError(String msg) {
    return Map.of("success", false, "error", msg);
}
```

所有 API 回應使用統一格式，前端可先判斷 `success` 再取 `data` 或 `error`。

六、Postman 測試指南

6.1 環境設定

項目	值
Base URL	<code>http://localhost:8080/mvjpars0626/api</code>
Content-Type	<code>application/json</code>
Accept	<code>application/json</code>

啟動 Tomcat 後，確認可瀏覽器開啟 `http://localhost:8080/mvjpars0626/api/employees` 確認伺服器運作正常。

6.2 測試案例

測試 1：POST 新增員工

```
POST http://localhost:8080/mvjpars0626/api/employees
Content-Type: application/json
```

```
{
  "name": "張三",
  "email": "zhangsan@test.com",
  "department": "IT",
  "salary": 65000,
  "hireDate": "2024-06-01"
}
```

預期回應：

- Status: 201 Created
- Header: Location: http://localhost:8080/mvjpars0626/api/employees/1
- Body:

```
{
  "success": true,
  "data": {
    "id": 1,
    "name": "張三",
    "email": "zhangsan@test.com",
    "department": "IT",
    "salary": 65000.0,
    "hireDate": "2024-06-01",
    "createdAt": "2024-06-26 14:30:00",
    "updatedAt": "2024-06-26 14:30:00"
  }
}
```

測試 2 : POST 新增更多測試資料

```
POST http://localhost:8080/mvjpars0626/api/employees
Content-Type: application/json
```

```
{
  "name": "李四",
  "email": "lisi@test.com",
  "department": "HR",
  "salary": 55000,
  "hireDate": "2023-03-15"
}
```

```
POST http://localhost:8080/mvjpars0626/api/employees
Content-Type: application/json
```

```
{
  "name": "王五",
  "email": "wangwu@test.com",
  "department": "IT",
  "salary": 72000,
  "hireDate": "2025-01-10"
}
```

測試 3 : GET 查詢全部 (分頁)

```
GET http://localhost:8080/mvjpars0626/api/employees?page=1&size=10
```

預期回應：

- Status: 200 OK
- Body: {"success":true,"data":[...]}

測試 4 : GET 依部門篩選

```
GET http://localhost:8080/mvjpars0626/api/employees?dept=IT
```

預期回應：只回傳 department 為 IT 的員工列表。

測試 5 : GET 查詢單筆

```
GET http://localhost:8080/mvjpars0626/api/employees/1
```

預期回應：

- Status: 200 OK
 - Body: {"success":true,"data":{"id":1,"name":"張三",...}}
-

測試 6 : GET 查詢不存在的 ID

```
GET http://localhost:8080/mvjpars0626/api/employees/999
```

預期回應：

- Status: 404 Not Found
 - Body: {"success":false,"error":"Not found: 999"}
-

測試 7 : PUT 修改員工

```
PUT http://localhost:8080/mvjpars0626/api/employees/1  
Content-Type: application/json
```

```
{  
  "name": "張三豐",  
  "email": "zhangsanfeng@test.com",  
  "department": "IT",  
  "salary": 75000,  
  "hireDate": "2024-06-01"  
}
```

預期回應：

- Status: 200 OK
 - Body: {"success":true,"data":{"id":1,"name":"張三豐","salary":75000.0,...}}
-

測試 8 : DELETE 刪除員工

```
DELETE http://localhost:8080/mvjpars0626/api/employees/3
```


預期回應：

- Status: 204 No Content
- Body: (空)

測試 9：POST 驗證錯誤 (name 為空)

```
POST http://localhost:8080/mvjpars0626/api/employees
Content-Type: application/json
```

```
{
  "email": "test@test.com",
  "department": "IT"
}
```

預期回應：

- Status: 400 Bad Request
- Body: {"success":false,"error":"Name is required"}

6.3 Postman Collection 匯入建議

可在 Postman 建立 Collection，命名為 mvjpars0626 API，並依序建立以下請求：

```
mvjpars0626 API
├─ POST 新增員工      POST /api/employees
├─ GET 查詢全部       GET /api/employees
├─ GET 依部門查詢     GET /api/employees?dept=IT
├─ GET 查詢單筆       GET /api/employees/1
├─ GET 查詢不存在     GET /api/employees/999
├─ PUT 修改員工       PUT /api/employees/1
├─ DELETE 刪除員工    DELETE /api/employees/3
└─ POST 驗證錯誤      POST /api/employees (缺 name)
```

建議設定 Collection Variable：

Variable	Initial Value	Current Value
base_url	http://localhost:8080/mvjpars0626/api	http://localhost:8080/mvjpars0626/api

七、開發流程速查

7.1 環境需求

工具	版本	用途
JDK	21+	編譯與執行
Maven	3.9+	建置工具
Tomcat	10.1+	Servlet 容器 (須支援 Jakarta EE 10)
MySQL	8.0+	資料庫
Postman	最新版	API 測試

7.2 部屬步驟

```
# 1. 確認 MySQL 已啟動, 並建立資料庫與表格
mysql -u root -p < create_table.sql

# 2. 使用 Maven 打包
mvn clean package

# 3. 將 target/mvjpars0626.war 複製到 Tomcat 的 webapps/
cp target/mvjpars0626.war /path/to/tomcat/webapps/

# 4. 啟動 Tomcat
catalina.bat run      # Windows
# catalina.sh run     # Linux/Mac

# 5. 驗證部署
curl http://localhost:8080/mvjpars0626/api/employees
```

7.3 常用 Maven 指令

指令	說明
<code>mvn clean</code>	清除 <code>target/</code>
<code>mvn compile</code>	編譯 Java 原始碼
<code>mvn package</code>	打包成 WAR
<code>mvn clean package</code>	清除後重新打包

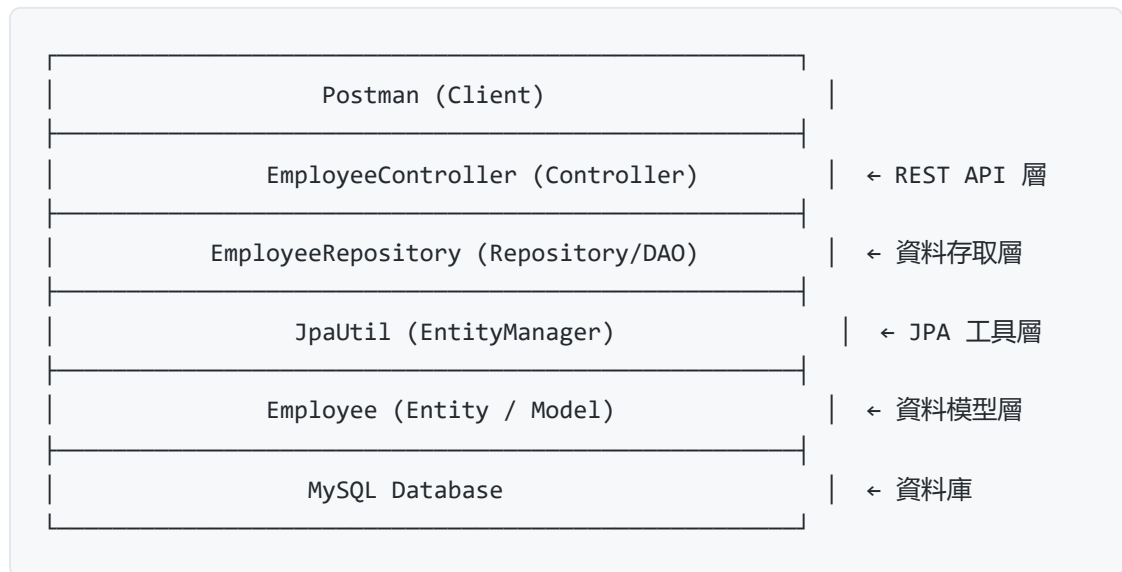
指令	說明
----	----

```
mvn dependency:tree
```

檢視相依套件樹

八、架構設計重點總結

分層架構



資料流範例 (POST 新增員工)

```
Postman → JSON Body
↓
EmployeeController.create()
  ↓ 1. 驗證 name 不為空
  ↓ 2. 呼叫 repo.save(emp)
  ↓
EmployeeRepository.save()
  ↓ 1. em.getTransaction().begin()
  ↓ 2. em.persist(emp)           → INSERT SQL
  ↓ 3. tx.commit()
  ↓ 4. em.close()
  ↓
回傳 emp (含自動生成的 id)
  ↓
EmployeeController
  ↓ 1. 產生 Location header
  ↓ 2. 回傳 201 + JSON
  ↓
Postman ← 201 Created
```

關鍵設計決策

決策	理由
<code>RESOURCE_LOCAL</code> 交易	簡化設定，適合無 EJB 容器的 Tomcat 環境
每次請求建立/關閉 EntityManager	Thread-safe，避免資源洩漏
<code>@PrePersist</code> / <code>@PreUpdate</code>	自動管理時間戳，業務邏輯簡潔
統一 JSON 回應格式 (<code>success</code> + <code>data</code> / <code>error</code>)	前端錯誤處理一致
泛型 <code>MyRepository<T, ID></code>	可重複使用，擴充其他 Entity 時不需重寫 CRUD

文件產生日期：2026-06-27 如有疑問或需補充，請聯絡開發團隊。